

lab4

要求：获取助教提供的新的xv6-k210-template；完成进程调度算法 `test_proc_rr`
`test_proc_priority` `test_proc_mlfq`

▼ 前置操作（切换新的仓库）

- 按照助教给的教程，从最下面的“系统性讲解（第一次必看！）”的第一个视频看下去就很直观了。
- 或者我这里也有一些视频抄送版本：
 - 退出你在lab1-3的仓库，目前我的位置是 `ubuntu@os-lab:~`，直接在这个目录下运行指令，将template克隆下来。然后别进到这个目录里，我修改了文件名为 `os-part4`。

```
git clone https://gitlab.eduxiji.net/pku230121066  
6/xv6-k210-template.git
```

- 进入目录并执行一个fetch

```
cd os-part4  
  
git fetch origin
```

视频里还有一个指令 `git checkout part4-scheduler`，它的作用主要是切换当前我们编辑的代码为part4-scheduler这个分支的，这个在后面通过不同测试样例时是需要的。

- 接下来，在我们目前的part4-scheduler分支下，运行这个代码，进入到xv6中，如果观察os-part4的文件中，`Makefile`的最下面多了一个 `make run_test`，这个指令就是在本地运行测试样例的指令。如果添加一个参数，变成 `make run_test SCHEDULER_TYPE=PRIORITY`，这样就可以挨个样例测试了

```
docker run -ti --rm -v ./:/xv6 -w /xv6 --privilege  
d=true docker.educg.net/cg/os-contest:2024p6 /bin/  
bash  
make run_test
```

这里我在 `.bashrc` 中添加了一个简化的指令，它可以直接定位到lab4所在的位置并且进入xv6，保存以后删除当前的powershell再重新开一个，直接输入 `inpart4` 就能进入xv6了

```
alias inpart4='cd os-part4 && docker run -ti --rm
-v ./:/xv6 -w /xv6 --privileged=true docker.educg.
net/cg/os-contest:2024p6 /bin/bash'
```

4. 助教说这个part4建议我们用不同的branch来保护，先观察上一步测试样例的反馈

```
Starting test program: test_proc_rr
Scheduler type: Round Robin
init: starting test_proc_rr
pid 3 test_proc_rr: unknown sys call 54322
pid 4 test_proc_rr: unknown sys call 54322
pid 5 test_proc_rr: unknown sys call 54322
testing output size:152, contents:
Testing RR Scheduler - Basic
RR Scheduler Process 3 completed
RR Scheduler Process 2 completed
RR Scheduler Process 1 completed
RR Basic Test Completed
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test1 output:
Testing RR Scheduler - Basic
RR Scheduler Process 3 completed
RR Scheduler Process 2 completed
RR Scheduler Process 1 completed
RR Basic Test Completed
Expected order: 3 2 1 0 0
Test1 PASSED
SCORE: 1
init: judger completed
make[1]: Leaving directory '/xv6'
```

可以看到Test1失败了，因为有一个未知的系统调用54322，接下来的步骤实际上和之前lab做的很像，我们查看 `sysnum.h` 中的定义，有 `#define SYS_set_timeslice 54322`，但 `syscall.c` 中没有对其进行注册，所以注册以后，我们给一个简单的占位函数，把它放在 `syscall.c` 下面（注意这里你打开的文件应该来自于 **os-lab4**），这里的 `timeslice` 也需要在进程定义中声明

```
uint64
sys_set_timeslice(void)
{
    int timeslice;
    struct proc *p = myproc();

    if (argint(0, &timeslice) < 0)
    {
        return -1;
    }

    if (timeslice < 0)
    {
        return -1;
    }

    p->timeslice = timeslice;

    return 0;
}
```

```
// Per-process state
struct proc
{
    ...
    int tmask;                // trace mask
    int timeslice;
};
```

5. 再 `make run_test` 就可以看到虽然说没通过测试点，但已经没有 `unknown syscall` 了。

6. 接下来，为了这个rr的测试点，建立一个branch，如果运行下面这个指令，现在你应该在新建好的这个branch上了。这个指令的意思是在当前这个branch的基础下新建一个分支并切换过去，所以如果我们还要建part4-priority分支，那就切换回一开始的part4-scheduler分支再建立就行。

```
git checkout -b part4-rr
```

这时候你可以运行 `git branch`，可以看到在刚刚创建的branch前面有一个*

7. 然后去希冀平台的gitlab，我们建一个新的仓库，并选择public权限，去掉README这一项，建好以后，复制下面这个代码，最后的url部分可以从gitlab的仓库中，点击clone按钮直接复制，应以https开头

```
git remote set-url origin https://gitlab.eduxiji.net/pkuxxxxxxxxxx/os-part4.git
git remote -v
```

然后你运行 `git remote -v` 就可以看到现在已经切换到了你自己的仓库了

8. 运行下面的代码，将这些改动push到你的仓库中的part4-rr分支，接下来，之后的push就只需要git push，而不用设置这条的origin了；如果在之后的提交中，希望切换到另一个分支，并push到另一个分支上，记得还是需要在第一次push时设置一下。

```
git add .
git commit -m "your comment"
git push -u origin part4-test
```

9. 提交的时候需要在校园网下进入这个网页<http://10.129.81.45:8080/>，在这次的lab中选择part4，填写好要交的分支名字与gitlab的网址，提交就行了。想查看提交记录，就进入查询页面，选择“用户（uid）”，输入自己的学号就能查得到。

▼ test_proc_rr

- 看一下测试样例：

```
#define ITERATIONS 1000000
#define LOOP 100
```

```

void task(int id) {
    volatile long long count = 0;
    for(int loop = 0; loop < LOOP; loop++) {
        for(int i = 0; i < ITERATIONS; i++) {
            count += (long long)i * (long long) i;
            if(i % (ITERATIONS/2) == 0) {
                // printf("RR Scheduler Process %d: i
iteration %d\n", id, i);
            }
        }
    }

    char buffer[50];
    int pos = 0;
    const char* parts[] = {"RR Scheduler Process ",
"0", " completed\n"};
    for (int i = 0; parts[0][i] != '\0'; i++) {
        buffer[pos++] = parts[0][i];
    }
    buffer[pos++] = '0' + id;
    for (int i = 0; parts[2][i] != '\0'; i++) {
        buffer[pos++] = parts[2][i];
    }

    write(1, buffer, pos);
}

```

```

/*
* Desc:
* We fork three processes, and set timeslice 1 to P1,
2 to P2, 3 to P3.
* The three processes just do the same work, and will
last a long enough time
* to encounter several timer interrupts.
*
* Expected:

```

```
* P3 will finish the job first, then P2, and P1 is the last.  
* The judge program will check the appearance and order of `Process {\d} completed`  
* to make sure you have implemented the Round-Robin algorithm with given timeslice.  
*/
```

```
int main() {  
    printf("Testing RR Scheduler - Basic\n");  
  
    int pid1, pid2, pid3;  
    if((pid1=fork())==0) {  
        set_timeslice(1);  
        task(1);  
        exit(0);  
    }  
    if((pid2=fork())==0) {  
        set_timeslice(2);  
        task(2);  
        exit(0);  
    }  
    if((pid3=fork())==0) {  
        set_timeslice(3);  
        task(3);  
        exit(0);  
    }  
    wait(0);  
    wait(0);  
    wait(0);  
  
    printf("RR Basic Test Completed\n");  
    exit(0);  
}
```

三个进程都进行了同一个任务，但是三个进程分配到的时间片不一样大，其中进程3的时间片分配最多，理论上来说应该是task3 → task2 → task1的顺序。

但如果只是在内核中把 `sys_set_timeslice()` 实现了，现在的顺序仍然是123的初始fork顺序。

```
init: test execution completed, starting judger
Judger: Starting evaluation
Test1 output:
Testing RR Scheduler - Basic
RR Scheduler Process 1 completed
RR Scheduler Process 2 completed
RR Scheduler Process 3 completed
RR Basic Test Completed
Expected order: 3 2 1 0 0
Test1 FAILED
SCORE: 0
```

- 在kernel/main.c中，我们得到了CPU 进行完了所有的初始化工作（页表、陷入机制、并发锁、设备、磁盘等）后，最后一行代码就是调用scheduler()。

```
// Per-CPU process scheduler.
// Each CPU calls scheduler() after setting itself u
p.
// Scheduler never returns. It loops, doing:
// - choose a process to run.
// - swtch to start running that process.
// - eventually that process transfers control
//   via swtch back to the scheduler.
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();           // 获取当前
    执行这段代码的 CPU 结构体
    extern pagetable_t kernel_pagetable; // 声明全局
    的内核页表

    c->proc = 0;                        // 初始化，
    当前 CPU 没有运行任何进程
    for (;;)                            // 经典的无
    限循环：调度器永远在工作
```

```

{
    // Avoid deadlock by ensuring that devices can in
    // interrupt.
    intr_on(); // 开启中
    断。非常重要！如果当前没有进程可运行，必须允许响应时钟/设备中
    断，否则系统就死锁了。

    int found = 0; // 标记这一
    轮遍历是否找到了可运行的进程
    for (p = proc; p < &proc[NPROC]; p++) // 遍历整个
    进程表（默认为 64 个槽位）
    {
        acquire(&p->lock); // 在读取/修
        改进程状态前，必须加锁，防止多核资源竞争

        if (p->state == RUNNABLE) // 【关键】
        看看这个进程是不是“准备好运行”了？
        {
            // 找到了一个可运行的进程！
            p->state = RUNNING; // 把状态改
            为“正在运行”
            c->proc = p; // 登记：当
            前 CPU 正在运行进程 p

            // 切换到该进程专属的内核页表
            w_satp(MAKE_SATP(p->kpagetable));
            sfence_vma(); // 刷新 TLB
            缓存，确保新页表生效

            // 【核心动作：上下文切换】
            // 保存当前 CPU 的调度器上下文到 c->context，
            // 加载进程 p 的上下文 (p->context)，然后直接跳转到
            p 上次暂停的地方！
            switch(&c->context, &p->context);

            // =====
            // 【注意！】代码执行到上面那行 switch 时，就跳出去
            了！这里的进程已经开始运行了。

```



```

        // 等到未来的某个时刻，该进程调用了 yield() 或 sleep() 放弃 CPU，
        // 会发生反向的 switch()，代码才会重新回到下面这一行继续执行！
        // =====

        // 进程运行结束或挂起，回到了调度器：切换回系统全局的内核页表
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();

        c->proc = 0; // 登记：当前 CPU 又变空闲了
        found = 1; // 标记：这轮遍历我们至少切过去执行了一个进程
    }
    release(&p->lock); // 释放进程锁，让别的 CPU 也可以碰这个进程
}

// 如果遍历了一整圈进程表，一个准备好的进程都没找到（都在 wait/sleep 甚至全死光了）
if (found == 0)
{
    intr_on(); // 再次确保中断打开
    asm volatile("wfi"); // Wait For Interrupt: 让 CPU 进入低功耗休眠，直到下一个硬件中断（比如时钟滴答声）把它叫醒，再接着循环找进程。
}
}
}

```

每个 `cpu` 初始化以后都会永不返回地进入 `scheduler` 的死循环，不断寻找可以运行的进程，其中 `intr_on()` 会允许内核接受中断，防止死锁，这一步的原因可以这样解释：

▼ `intr_on()` 与死锁：

1. 灾难场景假设（如果没有 `intr_on()`）
假设系统中有 2 个进程 P1 和 P2 正在运行（假设我们只有 1 个 CPU）：
2. P1 想读取磁盘文件：P1 调用系统调用，陷入内核。在内核里它请求读取磁盘。因为磁盘很慢，P1 需要等待磁盘硬件读完数据，于是它调用 `sleep()` 把自己的状态设为 `SLEEPING`，并且让出 CPU（这会去调用 `sched()` 然后 `swtch()` 跳回 `scheduler()` 的 `for(;;)` 循环）。
P2 也想读磁盘：调度器收回 CPU 后，循环发现了 P2（它是 `RUNNABLE` 的），于是 `swtch()` 给 P2。P2 运行了一会儿，也想读磁盘，同样它也调用 `sleep()`，变成了 `SLEEPING` 状态，并让出 CPU 回到了调度器。
现在，可怕的情况出现了：整个系统里所有的进程（P1 和 P2）都处于 `SLEEPING` 状态！没有一个是 `RUNNABLE` 的！
3. 在 `scheduler` 遍历进程表时，它会把当前正接受检查的进程上锁（`acquire(&p->lock);`），但是现在没有允许中断，所以每个进程都是 `SLEEPING`，如果这个时候磁盘硬件读完数据，试图发送一个硬件中断告诉 cpu 可以唤醒 P1
4. 但是由于 P1 在之前的检查中上锁了，这个中断没办法对 P1 的状态进行修改，`scheduler` 不停地 `acquire` 和 `release` 进程锁，很多情况下 `context` 切换回来的那一瞬间这个锁可能都是关闭的，这样 P1 和 P2 就会陷入互相等待。cpu 的中断处理函数一直被屏蔽，就会陷入死锁。

找到一个 `RUNNABLE` 的进程以后，调度器将这个进程的状态改为 `RUNNING`，cpu 的当前运行进程改成这个进程 p，刷新为 p 的内核页表。此后调度器会运行 `swtch` 这个函数，它对应的是一个 `swtch.S` 汇编码，其主要的作用就是交换两个 `context`，完成一个偷天换日的过程：

▼ `swtch` 在调度过程中：

```
# Context switch
#
# void swtch(struct context *old, struct context
# *new);
#
# Save current registers in old. Load from new.

.globl swtch
```

```

swtch:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ld s1, 24(a1)
    ld s2, 32(a1)
    ld s3, 40(a1)
    ld s4, 48(a1)
    ld s5, 56(a1)
    ld s6, 64(a1)
    ld s7, 72(a1)
    ld s8, 80(a1)
    ld s9, 88(a1)
    ld s10, 96(a1)
    ld s11, 104(a1)

    ret

```

在调用 `swtch(&c->context, &p->context)` 时，这个汇编会将传入的第一个参数（即当前 `cpu` 正在使用的上下文）与 `cpu` 刚设置好的新的进程 `p` 的 `context` 进行交换，这样，现在 `cpu` 就开始使用调度器调度来的 `p` 的上下文了。

- 在ics中学习的x86框架中，执行 `call` 指令后，CPU会在硬件层面上将下一条指令的地址压栈，然后再跳转到目标函数，执行 `ret` 指令时，CPU则自动从栈顶弹出该指令并跳转。
- 在xv6的RISCV框架中，并没有这种自动压栈的硬件操作，执行函数调用时，CPU会把下一条指令的地址存入一个专门的寄存器 `ra`，然后执行跳转。而在执行 `ret` 指令时，实际上 `cpu` 做了这样一件事： `jalr x0,0(ra)`，也就是简单地跳转到了 `ra` 存的地址。

所以在 **p1** 与 **p2** 调度的过程中，我们看到：

1. scheduler第一次调度，发现 **p2** 是 `RUNNABLE` 的，此时调用 `swtch(&cpu->context, &p->context);` 会把当前 `cpu` 的物理寄存器 `ra` 的内容存到 `a0`（即第一个参数 `oldcontext`）的起始位置 `ra` 变量中，然后存其他的上下文。这时，存的 `ra` 实际上是内核的下一步，也就是 `scheduler` 函数中，调用 `swtch` 之后的下一步。
2. 完成这种“偷天换日”的技巧：把 `a1`（即被调度上来，新换入 `cpu` 中的 **p2** 的 `context`）的起始位置加载到了 `cpu` 的真正物理寄存器 `ra` 中，改变了 `scheduler` 函数中调用 `swtch` 函数之后本应该的返回指令，因此接下来 `ret` 的时候，就直接进入了另一个进程 **p2** 的运行中。现在我们记清楚：此时 `cpu->context` 中的 `ra` 变量，存的指令是 `scheduler` 函数中，调用完 `swtch` 的下一条指令。
3. 接下来，当这个进程 **p2** 的时间片结束了或是直接 `yield` 出去，交由内核进行调度时，会有这个过程：

```
// Give up the CPU for one scheduling round.
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

// Switch to scheduler. Must hold only p->lock
// and have changed proc->state. Saves and restores
// intena because intena is a property of this
```

```

// kernel thread, not this CPU. It should
// be proc->intena and proc->noff, but that would
// break in the few places where a lock is held
// but
// there's no process.
void sched(void)
{
    int intena;
    struct proc *p = myproc();

    if (!holding(&p->lock))
        panic("sched p->lock");
    if (mycpu()->noff != 1)
        panic("sched locks");
    if (p->state == RUNNING)
        panic("sched running");
    if (intr_get())
        panic("sched interruptible");

    intena = mycpu()->intena;
    swtch(&p->context, &mycpu()->context);
    mycpu()->intena = intena;
}

```

`yield` 完成 **p2** 锁的持有以及状态的改变，`sched` 负责检查工作并设置 `intena`，又回去调用 `swtch` 了。

4. 这时候，`swtch.S` 又进行一次调换，我们注意这里的传参顺序是这样的：`swtch(&p->context, &mycpu()->context);`，现在，**p2** 进程正在使用的物理寄存器 `ra` 内容被保存在它自己的 `context` 中的 `ra` 变量，然后当前 `cpu` 的 `context` 里之前在 `scheduler` 中调用并保存好的 `ra`，就换入了真实的物理寄存器 `ra`，那么下一步，就回到了我们在 `scheduler` 中，第一次调用 `swtch` 函数的下一步了。
5. 下一步，内核又把当前的页表指针，从进程专属的内核页表又换回了全局的内核页表了。自此，一次调度就结束了，然后 `scheduler` 因为死循环，进入下一次调度，在这次调度中，调度器发现 **p1** 是 `RUNNABLE` 的，那么 **p1** 就会从头执行一次这个过程。

- 接下来，我们看看这个初始的 `scheduler` 在做什么，它实际上就是很忠实地按进程创建的顺序来遍历进程表，找到一个 `RUNNABLE` 的就换上来了。但我们需要时间片长的进程3最先完成，这时候，我们来看一下除了进程自己调用 `yield`、`sleep` 以外，内核自己决定进行调度的过程：

```
void
usertrap(void)
{
    // printf("run in usertrap\n");
    int which_dev = 0;

    ...

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2)
        yield();

    usertrapret();
}

void
kerneltrap() {
    int which_dev = 0;

    ...

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING) {
        yield();
    }
    // the yield() may have caused some traps to occur,
    // so restore trap registers for use by kernelvec.
    S's sepc instruction.
    w_sepc(sepc);
    w_sstatus(sstatus);
}
```

在内核态处理中断，以及用户态处理中断时，这个 `which_dev == 2` 的判断，就是在看这次中断是否来自时钟引发的中断，如果这个中断的类型满足，那么内核调用 `yield`，完成一次调度。

- 受到目前助教在测试样例中的 `set_timeslice()` 启发，在现有的基础上，我们只需要对 `usertrap` 和 `kerneltrap` 的逻辑做一点调整就好。当前的逻辑中，所有进程的 `timeslice` 都被当成了1，如果一开始设置好这个进程的 `timeslice`，那就应该在每次时钟中断时，检查现有的时间片用了多少，还剩多少，因此在proc中再添加一个成员 `int ticks;`。

严谨一点，我们要在一个进程的全周期中，都添加有关这个变量的维护，包括 `allocproc` `fork` `freeproc`。

```
static struct proc *
allocproc(void)
{
    ...
    p->context.sp = p->kstack + PGSIZE;
    p->ticks = 0;
    p->timeslice = 0;

    return p;
}

int fork(void)
{
    ...
    np->timeslice = p->timeslice;
    np->ticks = 0;

    np->state = RUNNABLE;
    ...
}

static void
freeproc(struct proc *p)
{
    ...
    p->xstate = 0;
    p->ticks = 0;
```

```

    p->timeslice = 0;
    p->state = UNUSED;
}

```

接下来，在usertrap中，添加这个逻辑。

```

void usertrap(void)
{
    ...
    // give up the CPU if this is a timer interrupt.
    if (which_dev == 2)
    {
        if (p->timeslice > 0)
        {
            p->ticks++;
            if (p->ticks >= p->timeslice)
            {
                p->ticks = 0;
                yield();
            }
        }
        else
        {
            yield();
        }
        yield();
    }
    usertrapret();
}

void usertrapret(void)
{
    ...
    if (which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING)
    {
        if (myproc()->timeslice > 0)
        {

```



```

        myproc()->ticks++;
        if (myproc()->ticks >= myproc()->timeslice)
        {
            myproc()->ticks = 0;
            yield();
        }
    }
    else
        yield();
}

// the yield() may have caused some traps to occur,
// so restore trap registers for use by kernelvec.
S's sepc instruction.
w_sepc(sepc);
w_sstatus(sstatus);
}

```

你还要记得，如果在这次准备yield这个进程，要把它ticks数置为0。

这样就得到了这个样例的1分。

▼ test_proc_priority

- 记着运行 `git checkout part4-priority` 到你建的另一个分支上。现在你再看这些文档，就跟上一个测试样例完全没关系了，你做的修改全部在之前的分支上。这个分支是从 `part4-priority` 建立的。
- 运行这个，以只测试该节点

```
make run_test SCHEDULER_TYPE=PRIORITY
```

- 看一下要求：

```

/*
 * Desc:
 * We fork three processes, and set priority 10 to P1,
 * 20 to P2, 30 to P3.
 * The three processes just do the same work, and will
 * last a long enough time
 * to encounter timer interrupt and yield.
 *

```

```

* Expected:
* P1(with the highest priority) will finish the job f
irst, then P2, and P3 is the last.
* The judge program will check the appearance and ord
er of `Process {\d} completed`
* to make sure you have implemented the priority algo
rithm with different priorities.
*/

```

```

int main() {
    printf("Testing Priority Scheduler - Basic\n");

    int pid1, pid2, pid3;
    if((pid1=fork())==0) {
        set_priority(10);    // highest one
        task(1);
        exit(0);
    }
    if((pid2=fork())==0) {
        set_priority(20);
        task(2);
        exit(0);
    }
    if((pid3=fork())==0) {
        set_priority(30);
        task(3);
        exit(0);
    }
    wait(0);
    wait(0);
    wait(0);

    printf("Priority Basic Test Completed\n");
    exit(0);
}

```

依然是三个进程执行一样的任务，在这里根据优先级不同，`priority` 越小优先级越高，按此顺序完成执行即可。

- 还是先注册，在 `proc.h` 里给 `proc` 添加一个 `priority` 的成员，并实现 `sys_setpriority`，再严谨一点，除了管理 `priority` 的全周期，我们在 `fork` 中，也让子进程的优先级设置为和父进程一样的。

```
uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    p->priority = priority;
    return 0;
}
```

```
struct proc
{
    ...
    int tmask;                // trace mask
    int priority;             // process priority for
    priority scheduler
};
```

```
int fork(void)
{
    ...
    np->priority = p->priority; // set priority of the
    child same as parent

    np->state = RUNNABLE;
    ...
}
```

- 如果我们直接运行 `make run_test SCHEDULER_TYPE=PRIORITY`，这样大概率你会得到一个比较随机的结束情况，但如果试的次数足够多，应该有概率出现1/2/3的顺序。
- 在这个样例中，我们实际上不用对trap函数做什么调整，我们在 `scheduler` 中调整一下寻找下一个进程的逻辑即可。

```
void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    extern pagetable_t kernel_pagetable;

    c->proc = 0;
    for (;;)
    {
        // Avoid deadlock by ensuring that devices can in
        // terrupt.
        intr_on();

        int found = 0;
        int highest_pri = 1000000;

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->priority < highest_pri)
            {
                highest_pri = p->priority;
            }
            release(&p->lock);
        }

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->priority == highest_pri)
```

```

    {
        // Switch to chosen process. It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        // printf("[scheduler]found runnable proc with pid: %d\n", p->pid);
        p->state = RUNNING;
        c->proc = p;
        w_satp(MAKE_SATP(p->kpagetable));
        sfence_vma();
        swtch(&c->context, &p->context);
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();
        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;

        found = 1;
    }
    if (p->state == RUNNABLE)
    {
        found = 1; // 如果有一个RUNNABLE的进程，就不让cpu
        // 休息，虽然不能执行，但还要重新跑一次比较的循环
        // 除非系统真的没有进程了，这时就让CPU休息一下，等中断来唤醒它
    }
    release(&p->lock);
}
if (found == 0)
{
    intr_on();
    asm volatile("wfi");
}
}
}

```

这里会有个问题，比方说极限一点，在两个fork之间突然产生了一个新的优先级更高的进程，那么这个逻辑就会忽略掉这个进程而去找之前那个确定好的优先级的进程了。此外，这里没有选择pid作为筛选逻辑，因为当两个进程的优先级相同时，如果用pid筛选，就会出现另一个进程饿死的情况。Gemini给出的解释是：如果要彻底考虑这种情况，就需要建立链表，在每次调度时给一个大锁，然后挑出来优先级最高的并运行，再释放这个锁。不过即使遇到我们之前那个问题，新建的进程也只是错过了一次调度，并不会造成很大的损失。所以就先这么办吧。

▼ test_proc_mlfq

- 阅读助教的文档，我们发现这个任务完全不用真的实现一个什么队列，我们甚至可以“不使用队列，而是在上一个测试样例的实现基础上继续实现优先级变化的统计逻辑即可。”
- 直接在part4-priority为基础创建一个新的branch。
- 运行这个，以只测试该节点

```
make run_test SCHEDULER_TYPE=MLFQ
```

- 会直接出现这个字样，我们去实现一下要求的 `#define SYS_get_priority 54324`

```
Starting test program: test_proc_mlfq
Scheduler type: MLFQ
init: starting test_proc_mlf
pid 7 test_proc_mlfq: unknown sys call 54324
pid 4 test_proc_mlfq: unknown sys call 54324
pid 6 test_proc_mlfq: unknown sys call 54324
```

```
uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
```

```

    p->priority = priority;
    return 0;
}

uint64
sys_get_priority(void)
{
    struct proc *p = myproc();
    return p->priority;
}

```

再看看运行结果：

```

init: starting test_proc_mlfq
testing output size:469, contents:
Testing MLFQ Scheduler - Basic
MLFQ Scheduler Process 5 with initial priority 3 and
final priority 3 completed
MLFQ Scheduler Process 2 with initial priority 1 and
final priority 1 completed
MLFQ Scheduler Process 4 with initial priority 5 and
final priority 5 completed
MLFQ Scheduler Process 3 with initial priority 2 and
final priority 2 completed
MLFQ Scheduler Process 1 with initial priority 10 and
final priority 10 completed
MLFQ with Priorities Test Completed
init: process pid=2 exited
init: test execution completed, starting judger
Judger: Starting evaluation
Test3 output:
Testing MLFQ Scheduler - Basic
MLFQ Scheduler Process 5 with initial priority 3 and
final priority 3 completed
MLFQ Scheduler Process 2 with initial priority 1 and
final priority 1 completed
MLFQ Scheduler Process 4 with initial priority 5 and
final priority 5 completed

```

```
MLFQ Scheduler Process 3 with initial priority 2 and
final priority 2 completed
MLFQ Scheduler Process 1 with initial priority 10 and
final priority 10 completed
MLFQ with Priorities Test Completed
Expected order: 2 0 0 0 1
Test3 FAILED
SCORE: 0
```

还是没什么头绪

- 看看测试样例的要求:

```
... (前面就是一个安全的打印函数而已)
void cpu_intensive_task(int id, int priority) {
    volatile long long count = 0;
    for(int loop = 0; loop < LOOP*10; loop++) {
        for(int i = 0; i < ITERATIONS; i++) {
            count += (long long)i * (long long) i;
            if(i % (ITERATIONS/2) == 0) {
                // printf("CPU Process %d (prio %d):
iteration %d\n", id, priority, i);
            }
        }
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}

void io_intensive_task(int id, int priority) {
    for(int i = 0; i < 30; i++) {
        // printf("IO Process %d (prio %d): iteration
%d\n", id, priority, i);
        sleep(1); // Simulate I/O operation
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}
```



```

void mixed_task(int id, int priority) {
    volatile long long count = 0;
    for(int i = 0; i < 20; i++) {
        // Do some computation
        for(int j = 0; j < ITERATIONS/10; j++) {
            count += (long long)j * (long long) j;
        }
        // printf("Mixed Process %d (priority %d): it
eration %d\n", id, priority, i);
        sleep(1); // Some I/O
    }
    int final_prio = get_priority();
    write_mlfq_completion(id, priority, final_prio);
}

```

```

/*
* Desc:
* We fork five processes with different characteristics and priorities:
* Priority: smaller number = higher priority
* - P1: CPU-intensive, low priority (10) - should be demoted
* - P2: I/O-intensive, high priority (1) - should stay in high queues
* - P3: CPU-intensive, high priority (2) - initially high but may be demoted
* - P4: I/O-intensive, medium priority (5) - should be promoted
* - P5: Mixed workload, high priority (3)
*
* Expected:
* The highest priority I/O-bound processes (P2) should finish first,
* and the lowest priority CPU-bound (P1) should complete last.
* CPU-intensive process P1 & P3 will have an ending priority lower than initial,
* while I/O-intensive process P4 will have an higher

```

```

priority than the initial one.
*/

int main() {
    printf("Testing MLFQ Scheduler - Basic\n");

    int pid1, pid2, pid3, pid4, pid5;

    // Low priority CPU-bound (may be demoted)
    if((pid1=fork())==0) {
        set_priority(10);    // Lowest priority (large
                             // st number)
        cpu_intensive_task(1, 10);
        exit(0);
    }

    // Highest priority I/O-bound (should stay in high
    // queues)
    if((pid2=fork())==0) {
        set_priority(1);    // Highest priority (smallest
                             // number)
        io_intensive_task(2, 1);
        exit(0);
    }

    // High priority CPU-bound (initially high but may
    // be demoted)
    if((pid3=fork())==0) {
        set_priority(2);    // High priority
        cpu_intensive_task(3, 2);
        exit(0);
    }

    // Medium priority I/O-bound (initially low but may
    // be promoted)
    if((pid4=fork())==0) {
        set_priority(5);    // Medium priority
        io_intensive_task(4, 5);
    }
}

```

```

        exit(0);
    }

    // High priority mixed workload
    if((pid5=fork())==0) {
        set_priority(3);    // High priority
        mixed_task(5, 3);
        exit(0);
    }

    for (int i = 0; i < 5; i++) {
        wait(0);
    }

    printf("MLFQ with Priorities Test Completed\n");
    exit(0);
}

```

在这个测试样例中，有三种不同的任务：

1. cpu密集型：它会一直执行大量的乘法，永远不让出cpu，这个任务在mlfq的规则里，优先级应当很低。
2. io密集型：它只干一点活，然后就交出cpu，这是调度器很喜欢的任务，为了保证交互响应，它应当被保持高优先级或者提高它的优先级。
3. mixed任务：做一点乘法，然后sleep让出cpu，它的优先级应该属于中游水平。

按照文档的要求，我们最后应该有**P2**最先结束，而最低优先级的**P1**应最后完成。CPU密集型的**P1**和**P3**最终优先级会低于初始值，而I/O密集型的**P4**最终优先级会高于初始值。

- 阅读文档，这里io密集型任务调用sleep其实是在模拟cpu等待io，当累积时间/比例超过一定程度后就会获得优先级的提升，而cpu密集型，则会根据它被“强制下线”的次数来降低优先级。
 - 所以在proc结构体中，我们还要声明一些新的成员变量来满足这个判断过程。

```

struct proc
{

```

```

...
    int priority;                // process priority
for priority scheduler
    int now_priority;           // now priority for
mlfq scheduler
    int io_ticks;               // ticks doing I/O,
for mlfq scheduler
    int cpu_ticks;              // ticks doing CPU
work, for mlfq scheduler
};

```

`now_priority` 就是在所有进程的调度与运行中的增减以后的实时优先级。

`priority` 则是通过 `sys_setpriority()` 设定的初始优先级。

`io_ticks` 记录了这个进程做了多少个io任务的时间片（即 `sleep` /主动触发 `sched` 了多少次）。

`cpu_ticks` 记录了这个进程做了多少个cpu任务的时间片（即被强制下线了多少次）。

- 因此我们还得修改一些函数，包括进程全周期的维护函数，以及 `sys_getpriority` 和 `sys_setpriority`。

```

uint64
sys_set_priority(void)
{
    int priority;
    if (argint(0, &priority) < 0)
    {
        return -1;
    }
    struct proc *p = myproc();
    p->priority = priority;
    p->now_priority = priority; // set now_priority
same as initial priority
    return 0;
}

uint64
sys_get_priority(void)

```

```

{
    struct proc *p = myproc();
    return p->now_priority;
}

```

这里的 `fork` 有点要注意，新得到的子进程，它的优先级虽然与父进程相同，但它的 `io_ticks` 与 `cpu_ticks` 都要置为0，因为它没有经历过调度。

```

int fork(void)
{
    ...
    pid = np->pid;

    np->priority = p->priority; // set priority of the child same as parent

    np->now_priority = p->now_priority; // set now_priority of the child same as parent

    np->io_ticks = 0; // set io_ticks of the child to 0

    np->cpu_ticks = 0; // set cpu_ticks of the child to 0

    np->state = RUNNABLE;

    release(&np->lock);

    return pid;
}

```

- 接下来我们开始对这些进程进行赏罚机制。
 - 在之前我们知道如果一个进程被cpu强制下线，那么会由trap函数通过调用 `yield` 间接调用 `shed`，而在io密集型的任务中，我们则是直接调用了 `sleep` 来模拟。因此我们可以直接在 `yield` 以及 `sleep` 的函数上做文章。

- 我在这里选择的策略是一个进程每经过固定次数的被调度运行后，检查主动 `sleep` 与被动 `yield` 的比例，根据比例决定升降 `priority`，你也可以直接在 `sleep` 中检查 `io_ticks` 的次数并升高 `priority`。这里我新增了一个评测函数，每次 `sleep` 和 `yield` 时都进行一次检验。

```
void evaluate_priority(struct proc *p, int total_ticks)
{
    int cpu_percent = (p->cpu_ticks * 100) / total_ticks;

    // 判定规则：
    if (cpu_percent >= 70)
    {
        // CPU 动作占 70% 以上，典型的贪婪型 -> 降级
        if (p->now_priority < 100)
        {
            p->now_priority++;
        }
    }
    else if (cpu_percent <= 30)
    {
        // CPU 动作低于 30% (意味着 IO 占了 70% 以上)，典型的交互型 -> 升级
        if (p->now_priority > 1)
        {
            p->now_priority--;
        }
    }
    else
    {
        // CPU 占比在 30% 到 70% 之间 (混合型)，不奖不罚，维持原状
    }
    // 结算后重置计数器，开始新一轮的考核周期
    p->cpu_ticks = 0;
    p->io_ticks = 0;
}
```

在 `yield` 与 `sleep` 中添加相应逻辑:

```
void yield(void)
{
    struct proc *p = myproc();
    acquire(&p->lock);
    p->cpu_ticks += 1;
    int total_ticks = p->io_ticks + p->cpu_ticks;
    if (total_ticks >= 30)
    {
        evaluate_priority(p, total_ticks);
    }
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

void sleep(void *chan, struct spinlock *lk)
{
    struct proc *p = myproc();
    if (lk != &p->lock)
    {
        // DOC: sleeplock0
        acquire(&p->lock); // DOC: sleeplock1
        release(lk);
    }

    p->io_ticks += 1;
    int total_ticks = p->io_ticks + p->cpu_ticks;
    if (total_ticks >= 30)
    {
        evaluate_priority(p, total_ticks);
    }
    // Go to sleep.
    p->chan = chan;
    p->state = SLEEPING;
    sched();
    // Tidy up.
    p->chan = 0;
```

```

// Reacquire original lock.
if (lk != &p->lock)
{
    release(&p->lock);
    acquire(lk);
}
}

```

最后把 `scheduler` 里面关于 `priority` 的判断变成关于 `now_priority` 的判断，并添加一些有关 `base_priority` 的判断，就行了。

```

void scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    extern pagetable_t kernel_pagetable;

    c->proc = 0;
    for (;;)
    {
        // Avoid deadlock by ensuring that devices can
        interrupt.
        intr_on();

        int found = 0;
        int highest_pri = 1000000;
        int base_pri = 1000000;

        for (p = proc; p < &proc[NPROC]; p++)
        {
            acquire(&p->lock);
            if (p->state == RUNNABLE && p->now_priority
                < highest_pri)
            {
                highest_pri = p->now_priority;
                base_pri = p->priority;
            }
            else if (p->state == RUNNABLE && p->now_prio

```



```

rity == highest_pri && p->priority < base_pri)
{
    base_pri = p->priority;
}
release(&p->lock);
}

for (p = proc; p < &proc[NPROC]; p++)
{
    acquire(&p->lock);
    if (p->state == RUNNABLE && p->now_priority
== highest_pri && p->priority == base_pri)
    {
        // Switch to chosen process. It is the pr
ocess's job
        // to release its lock and then reacquire
it
        // before jumping back to us.
        // printf("[scheduler]found runnable proc
with pid: %d\n", p->pid);
        p->state = RUNNING;
        c->proc = p;
        w_satp(MAKE_SATP(p->kpagetable));
        sfence_vma();
        swtch(&c->context, &p->context);
        w_satp(MAKE_SATP(kernel_pagetable));
        sfence_vma();
        // Process is done running for now.
        // It should have changed its p->state bef
ore coming back.
        c->proc = 0;

        found = 1;
    }
    if (p->state == RUNNABLE)
    {
        found = 1; // 如果有一个RUNNABLE的进程，就不让
cpu休息，虽然不能执行，但还要重新跑一次比较的循环

```

```
        // 除非系统真的没有进程了，这时就让C
        PU休息一下，等中断来唤醒它
    }
    release(&p->lock);
}
if (found == 0)
{
    intr_on();
    asm volatile("wfi");
}
}
}
```